

TITLE:

Uncertainty-Aware Object Detection under Distribution Shift on Edge Hardware

<i>Contents</i>	<i>Page</i>
1. Introduction	1
2. Related Work	2
3. Methodology	4
4. Experimental Setup	6
5. Results	6
5.1 Uncertainty Under Occlusion	6
5.2 Uncertainty Under Lighting Variation	7
5.3 Latency on Laptop and Raspberry Pi 5	7
5.4 Development Timeline	8
6. Discussion and Limitations	8

7. Conclusion and Future Work	9
--------------------------------------	----------

8. References	12
----------------------	-----------

List of Figures	Page
Figure 1. Uncertainty vs. lighting condition	10
Figure 2. Uncertainty vs. occlusion level	10
Figure 3. Latency by condition and hardware	11
Figure 4. Development timeline (Oct 2025–Mar 2026)	11

OptoGuard: Uncertainty-Aware Object Detection under Distribution Shift on Edge Hardware

Technical report on Monte Carlo Dropout-based uncertainty analysis for YOLOv8n under distribution shift.

Author: Syed Suhaib Ahmed Khair

1. Introduction

Object detection systems such as YOLO have demonstrated strong performance on benchmark datasets and have become widely adopted across computer vision applications ranging from surveillance to robotics. However, their behaviour under real world deployment conditions is often less predictable than benchmark evaluations suggest. In particular, neural detectors frequently exhibit overconfident predictions even when inputs deviate from the training distribution. Variations in illumination, partial occlusion, sensor noise, or the presence of unfamiliar objects can substantially degrade prediction quality while confidence scores remain artificially high. As a result, incorrect detections may appear trustworthy to downstream systems despite the model operating under degraded conditions. Distribution shift occurs when the joint distribution of inputs and outputs differs between training and production data, often causing significant performance degradation. It breaks the core machine learning assumption that training/test data are independent and identically distributed.

This phenomenon becomes especially problematic in unattended deployments or embedded systems where a model operates continuously without human supervision. Edge devices installed in corridors, laboratories, or entry points often encounter dynamic environmental conditions such as fluctuating lighting, cluttered scenes, or objects not represented in the training dataset. When detectors remain overconfident under such conditions, failures may go unnoticed until they produce operational consequences. Addressing this reliability gap requires mechanisms that allow models to express uncertainty about their predictions rather than always reporting deterministic confidence values.

The OptoGuard project investigates this issue through a controlled experimental study focused on uncertainty estimation for lightweight object detectors. The system integrates the YOLOv8n detector with a Monte Carlo (MC) Dropout inference wrapper that enables stochastic forward passes at test time. Rather than relying solely on a single deterministic confidence score, the detector produces a distribution of predictions whose variability reflects how sensitive the model is to stochastic perturbations within its internal structure. The variance across predictions can therefore be interpreted as a practical proxy for epistemic uncertainty.

The study evaluates detector behaviour under five controlled conditions: baseline indoor lighting, altered illumination conditions, graded occlusion levels, exposure to novel out of distribution objects, and deployment on Raspberry Pi 5 hardware. For each frame and detection instance, the pipeline records the mean predicted confidence across stochastic passes together with the standard deviation of those predictions. These two quantities together provide a richer view of model behaviour: the mean represents the expected confidence of the detection while the standard deviation reflects the stability of that prediction under stochastic perturbation.

Importantly, this work does not attempt to introduce a new uncertainty estimation technique, but its primary contribution lies in systematically documenting how uncertainty estimates evolve under common environmental perturbations and in quantifying the computational overhead associated with stochastic inference on resource constrained edge hardware. Empirical analyses of this type provide an important foundation for more advanced techniques, including structured dropout variants such as DropBlock and adaptive learning systems that trigger retraining when uncertainty signals indicate distributional drift. By establishing how uncertainty behaves under simple real world disturbances, the project aims to clarify whether uncertainty estimation can serve as a practical monitoring signal for deployed vision systems.

2. Related Work

Uncertainty estimation via stochastic inference. The use of dropout during inference to approximate Bayesian posterior predictive distributions was formalised by Gal and Ghahramani. In this framework, dropout masks sampled at test time produce a set of stochastic predictions whose mean and variance approximate the predictive distribution of a Bayesian neural network. The variance of these predictions provides a practical estimate of epistemic uncertainty and has been widely adopted due to its minimal architectural modifications and straightforward implementation. Compared with fully Bayesian neural networks, which require complex posterior estimation procedures, Monte Carlo Dropout offers a computationally accessible approximation suitable for large deep learning models.

Within the context of object detection, uncertainty estimation has gained increasing attention as systems are deployed in safety sensitive environments. Stochastic inference allows detectors to capture the degree of disagreement between multiple forward passes,

which reflects uncertainty about the internal representation of the scene. Although this method does not provide a fully calibrated Bayesian posterior, it offers a tractable approach for estimating predictive variability without modifying the underlying architecture.

Subsequent work has explored spatially structured dropout mechanisms designed to better capture uncertainty in convolutional feature maps. For instance, Monte Carlo DropBlock introduces contiguous spatial masking rather than independent neuron level dropout, improving the representation of spatial uncertainty while preserving detection accuracy in certain scenarios. Structured dropout techniques aim to reflect the spatial correlations inherent in convolutional architectures, potentially yielding more informative uncertainty estimates in dense prediction tasks. Although such techniques may offer advantages for detection architectures, the present study employs standard MC Dropout for clarity and simplicity while treating structured variants as a potential extension.

Distribution shift and continual learning. Real world deployment environments rarely match the training distribution used to develop computer vision models. Changes in lighting, scene composition, object appearance, and camera position introduce distribution shifts that can degrade model performance over time. Continual learning approaches attempt to address this challenge by updating models incrementally while avoiding catastrophic forgetting of previously learned knowledge. Methods such as EvoCL demonstrate how adaptive mechanisms can maintain performance across evolving domains by selectively integrating new information into existing models.

Despite progress in continual learning techniques, determining when a model should adapt remains a challenging problem. Many systems rely on predefined retraining schedules or manual monitoring of accuracy metrics, which are often unavailable during deployment. Uncertainty signals offer a potential alternative mechanism for detecting when the model's internal representation no longer aligns with incoming data. Investigating whether uncertainty estimates respond reliably to environmental shifts therefore represents an important step toward building adaptive detection systems.

Edge deployment of vision models. Considerable research has been devoted to deploying deep learning models on resource constrained devices. Techniques such as quantisation, pruning, model compression, and resolution scaling have been widely studied to reduce computational requirements while maintaining acceptable accuracy. Edge deployment introduces additional constraints such as limited memory bandwidth, thermal restrictions, and restricted power consumption. These constraints often require careful balancing between model complexity and inference latency.

In contrast, uncertainty estimation methods have received comparatively limited attention in edge contexts, partly because stochastic inference increases computational cost. Running multiple forward passes per frame inherently multiplies inference time, raising concerns about real time performance. Understanding the magnitude of this overhead is therefore important when designing systems intended for deployment on devices such as the Raspberry Pi. Empirical measurements of latency and stability under edge hardware

constraints remain relatively scarce in the literature, motivating the evaluation performed in this work.

3. Methodology

3.1 Base Detector and Uncertainty Estimator

The experiments use YOLOv8n from the Ultralytics framework due to its compact architecture and suitability for edge deployment. The model represents a lightweight variant of the YOLOv8 architecture, designed to maintain acceptable detection performance while reducing computational cost. Its relatively small parameter count makes it particularly appropriate for embedded systems where memory and processing resources are limited.

Let $f_{\theta}(x)$ denote the detector parameterised by θ with dropout layers represented by D . Standard inference produces bounding boxes, class predictions, and scalar confidence scores after non maximum suppression (NMS). These confidence values typically represent the model's estimated probability that a detected object belongs to a particular class.

To estimate predictive uncertainty, inference is performed using Monte Carlo Dropout. During inference the model remains in training mode so that dropout layers remain active, while gradient computation is disabled to avoid unnecessary computational overhead. Maintaining dropout during inference introduces stochastic perturbations into the network's intermediate activations, producing slightly different predictions across repeated passes of the same input.

For each input frame x , T stochastic forward passes are executed:

$$\hat{y}^t = f_{\{\theta, D_t\}}(x), t = 1, \dots, T$$

where D_t denotes the dropout mask sampled during pass t .

After NMS is applied to each pass independently, detections are associated across passes based on spatial proximity and predicted class labels. Bounding boxes that overlap within a predefined threshold are grouped together and treated as instances of the same underlying detection. This grouping step ensures that predictions corresponding to the same object are aggregated when computing uncertainty statistics.

For each detection group, the resulting set of confidence values $\{p^t\}$ is used to compute the mean confidence μ and standard deviation σ :

$$\mu = (1/T) \sum p^t$$

$$\sigma = \sqrt{(1/(T-1)) \sum (p^t - \mu)^2}$$

The mean μ replaces the original confidence score while σ serves as an uncertainty proxy reflecting the dispersion of predictions across stochastic passes. Larger σ values indicate

that the detector's predictions are unstable under dropout perturbations, suggesting greater epistemic uncertainty.

In practice $T=30$ stochastic passes are used for laptop experiments, whereas $T=20$ is used for Raspberry Pi deployment in order to balance estimation stability against latency. Reducing the number of passes decreases the precision of the variance estimate but significantly improves runtime performance on constrained hardware.

3.2 Experimental Conditions

Experiments were conducted using a fixed webcam observing a desk scale scene containing common household objects. Environmental conditions were manipulated while keeping the detector configuration unchanged. Maintaining a consistent camera position ensures that variations in detection behaviour can be attributed primarily to environmental changes rather than viewpoint differences.

Baseline measurements were recorded under normal indoor lighting conditions using familiar objects such as bottles, mobile phones, notebooks, and laptop devices. This scenario provides the reference distribution for both uncertainty and latency measurements against which other experimental conditions are compared.

Lighting perturbations were introduced through two distinct conditions: reduced ambient illumination and strong backlighting positioned behind the objects. Reduced lighting conditions simulate environments where ambient brightness is insufficient for clear image capture, while backlighting produces high contrast silhouettes that can disrupt edge detection and feature extraction. Both conditions represent common challenges in real world indoor deployments.

Occlusion experiments were conducted by partially covering objects using books or hands. Three approximate occlusion levels—25%, 50%, and 75%—were applied to evaluate how increasing loss of visual information affects prediction stability. Although the occlusion percentages were estimated visually rather than measured precisely, the same approximate levels were maintained across experimental runs to ensure consistency.

Out of distribution object scenarios were created by introducing objects unlikely to appear in the COCO dataset, including unusual toys, abstract shapes, and uncommon household items. These inputs test the model's behaviour when confronted with unfamiliar visual categories that lie outside the training distribution.

Finally, the baseline configuration was replicated on a Raspberry Pi 5 device using identical model weights and inference code. The number of stochastic passes and input resolution were adjusted when necessary to maintain feasible execution times. This setup enables direct comparison of uncertainty behaviour and latency between a standard laptop environment and a representative edge computing platform.

For every frame and detection instance, the pipeline records the following attributes: experimental condition, frame index, predicted class, mean confidence μ , confidence

standard deviation σ , inference latency in milliseconds, hardware platform, and additional condition specific annotations. All measurements are stored as structured CSV files within the project repository to facilitate reproducibility and post hoc analysis.

4. Experimental Setup

Experiments were conducted on two hardware platforms: a laptop with 16 GB of RAM and a Raspberry Pi 5 equipped with 8 GB of memory. The Raspberry Pi platform was selected because it represents a widely used edge computing device capable of running modern deep learning models under constrained computational resources.

To maintain comparability between systems, inference on the laptop was restricted to CPU execution rather than GPU acceleration. This ensures that both platforms represent CPU only inference scenarios typical of edge deployments. Although GPUs could significantly reduce runtime, excluding them allows the study to focus on conditions more representative of embedded environments.

The software environment was managed using a Python virtual environment defined through a shared requirements.txt configuration file. The same environment specification was reproduced on the Raspberry Pi to minimise dependency related discrepancies between platforms. Consistent library versions are important for ensuring reproducibility of results and preventing subtle differences in inference behaviour across systems.

Data collection followed a semi structured protocol. For each experimental condition, between approximately 50 and 200 frames were captured depending on the time required to manipulate physical scene elements such as lighting or occluding objects. Although the dataset is relatively small compared with standard computer vision benchmarks, it is sufficient to observe consistent trends in uncertainty behaviour across conditions.

Multiple sessions were conducted over the course of development, and experimental runs were repeated whenever modifications to the detection pipeline required updated measurements. This iterative process helped ensure that results reflected the current implementation rather than outdated experimental configurations.

All analysis procedures were implemented in a Jupyter notebook environment. The analysis workflow aggregates CSV files generated during experiments, applies basic filtering to remove frames without detections, and produces visualisations examining uncertainty behaviour across occlusion levels, lighting conditions, hardware platforms, and development stages.

5. Results

5.1 Uncertainty Under Occlusion

Experimental results demonstrate a consistent increase in uncertainty as the degree of occlusion increases. Moving from approximately 25% to 50% occlusion produces a substantial rise in σ values, indicating greater variability across stochastic predictions. The increase from 50% to 75% occlusion remains observable but is comparatively smaller, suggesting that once a large portion of the object becomes hidden, the detector's internal representation has already degraded significantly.

This behaviour likely reflects the manner in which convolutional detectors construct object representations from spatial features. When only a small portion of the object is obscured, sufficient visual information remains for the network to produce stable predictions. However, once a large portion of the object's structure is removed, the feature representation becomes ambiguous, leading different dropout masks to produce divergent predictions.

Importantly, this pattern appears across multiple object categories present in the scene rather than being restricted to a single object type. The consistency of this behaviour suggests that uncertainty estimates respond systematically to the loss of spatial information within the detector's feature representations rather than reflecting class specific idiosyncrasies.

5.2 Uncertainty Under Lighting Variation

Both reduced illumination and strong backlighting conditions produce elevated uncertainty relative to the baseline scenario. Despite representing different types of visual degradation, the resulting uncertainty magnitudes are similar. Reduced illumination diminishes global texture and colour information, whereas backlighting primarily affects edge contrast and silhouette visibility.

The similarity in σ values suggests that the uncertainty metric primarily reflects the overall severity of perceptual degradation rather than its specific cause. From a monitoring perspective, this behaviour may be advantageous because a single uncertainty threshold could detect multiple categories of environmental degradation without requiring condition specific calibration.

These results also highlight the importance of considering environmental robustness when deploying vision systems. Lighting variations are among the most common sources of distribution shift in real world environments, and detectors trained on curated datasets often struggle when illumination conditions differ from those present during training.

5.3 Latency on Laptop and Raspberry Pi 5

Latency measurements reveal clear differences between the two hardware platforms. On the laptop, inference latency remains relatively stable across experimental conditions. Variation is primarily driven by the number of objects detected within each frame, which

influences the cost of post processing operations such as detection grouping and statistical aggregation.

When executed with $T=30$ stochastic passes, the laptop system maintains a processing rate compatible with near real time monitoring applications. Although stochastic inference increases computation relative to standard single pass inference, the additional cost remains manageable for moderate frame rates.

In contrast, the Raspberry Pi exhibits substantially higher latency and greater variability. Thermal effects, limited CPU resources, and operating system scheduling contribute to fluctuations in processing time. Even so, reducing the number of stochastic passes to $T=20$ allows the Raspberry Pi to perform inference reliably, albeit at a lower throughput.

This level of performance is suitable primarily for intermittent monitoring scenarios rather than continuous high frame rate analysis. For example, applications such as periodic scene inspection or event triggered analysis could accommodate the observed latency characteristics.

5.4 Development Timeline

The project development timeline spans approximately six months. Early stages focused on implementing the stochastic inference wrapper and establishing reliable data logging mechanisms. Subsequent phases involved refining detection grouping logic, improving experimental repeatability, and stabilising the codebase.

During intermediate development stages, several revisions were required to address inconsistencies in detection grouping across stochastic passes. Correctly associating detections produced by different dropout masks proved essential for generating meaningful uncertainty statistics.

Later experiments show more consistent uncertainty measurements and improved latency stability as the system architecture matured. The timeline therefore reflects the iterative engineering process underlying the study rather than representing a controlled experimental variable.

6. Discussion and Limitations

The experiments indicate that MC Dropout provides a useful uncertainty signal for object detection systems operating under environmental perturbations. As visual conditions deteriorate due to occlusion or lighting variation, the variability across stochastic predictions increases in a consistent and measurable manner. This behaviour suggests that uncertainty estimates could function as practical monitoring indicators capable of signalling degraded operating conditions in deployed systems.

The cross platform comparison also reveals that uncertainty behaviour remains qualitatively similar between laptop and edge hardware deployments. This observation

implies that uncertainty thresholds calibrated during development may remain meaningful when transferred to resource constrained devices.

However, several limitations constrain the interpretation of these results. All experiments rely on COCO pretrained weights without environment specific fine tuning, meaning that measured uncertainty may partially reflect general distribution mismatch rather than solely the manipulated experimental conditions.

Occlusion levels and lighting intensities were estimated visually rather than measured quantitatively, limiting the precision of experimental control. Future experiments could incorporate controlled illumination measurements or synthetic occlusion masks to improve repeatability.

The dataset is also geographically and contextually narrow, consisting primarily of a single indoor desk environment. Broader evaluation across multiple scenes, camera angles, and environmental conditions would be necessary to establish stronger generalisation claims.

Furthermore, the study evaluates only one uncertainty estimation technique and does not include comparisons with alternative approaches such as deep ensembles, structured dropout variants, or post hoc calibration methods. These limitations restrict the extent to which broader conclusions can be drawn regarding optimal uncertainty estimation strategies for object detection.

7. Conclusion and Future Work

This study examines the behaviour of Monte Carlo Dropout uncertainty estimates within a lightweight YOLO based object detection pipeline operating under real world perturbations and edge deployment constraints. Experimental results demonstrate that predictive uncertainty increases systematically in response to occlusion and lighting degradation while remaining observable on both laptop and Raspberry Pi hardware.

Although stochastic inference introduces additional computational overhead, the resulting latency remains manageable for certain low throughput monitoring scenarios on edge devices. These findings support the use of uncertainty estimation as a practical diagnostic signal for deployed vision systems, particularly in contexts where ground truth labels are unavailable during operation.

Future work may extend this investigation by comparing MC Dropout with spatially structured uncertainty methods such as Monte Carlo DropBlock, exploring ensemble based uncertainty estimation strategies, and developing adaptive mechanisms that trigger model updates when sustained increases in uncertainty indicate distributional drift.

Additional research directions include evaluating uncertainty based monitoring in outdoor environments, integrating calibration techniques that map uncertainty statistics to probabilistic confidence estimates, and exploring lightweight optimisation strategies that reduce the computational cost of stochastic inference on edge devices. Such developments

could help transform uncertainty estimation from a diagnostic tool into a core component of reliable real world vision systems.

Figures

Figure 1. Uncertainty vs. lighting condition

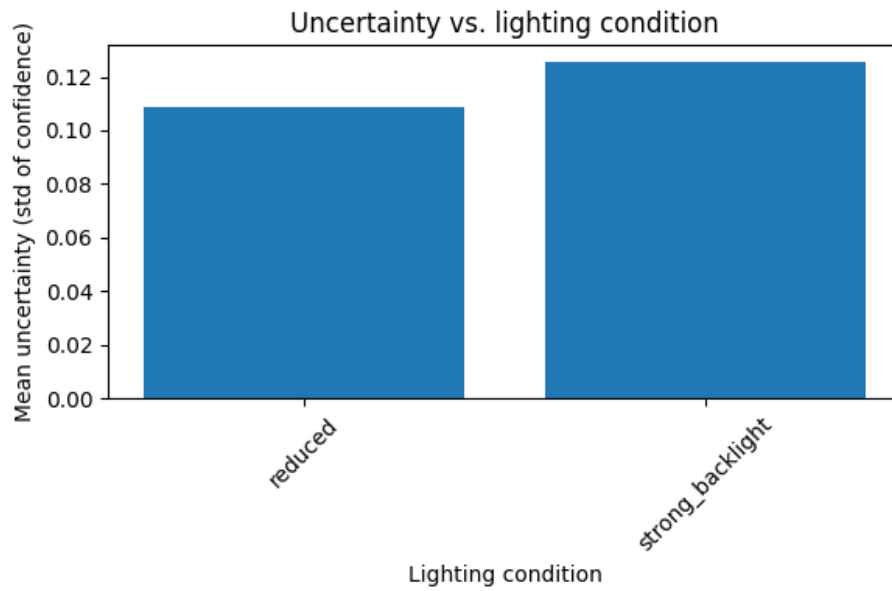


Figure 2. Uncertainty vs. occlusion level

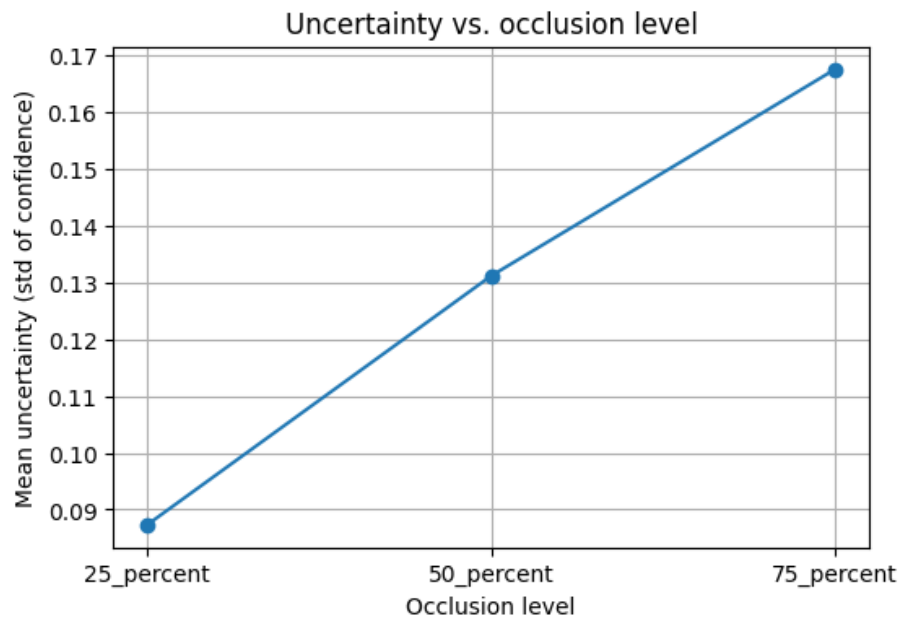


Figure 3. Latency by condition and hardware

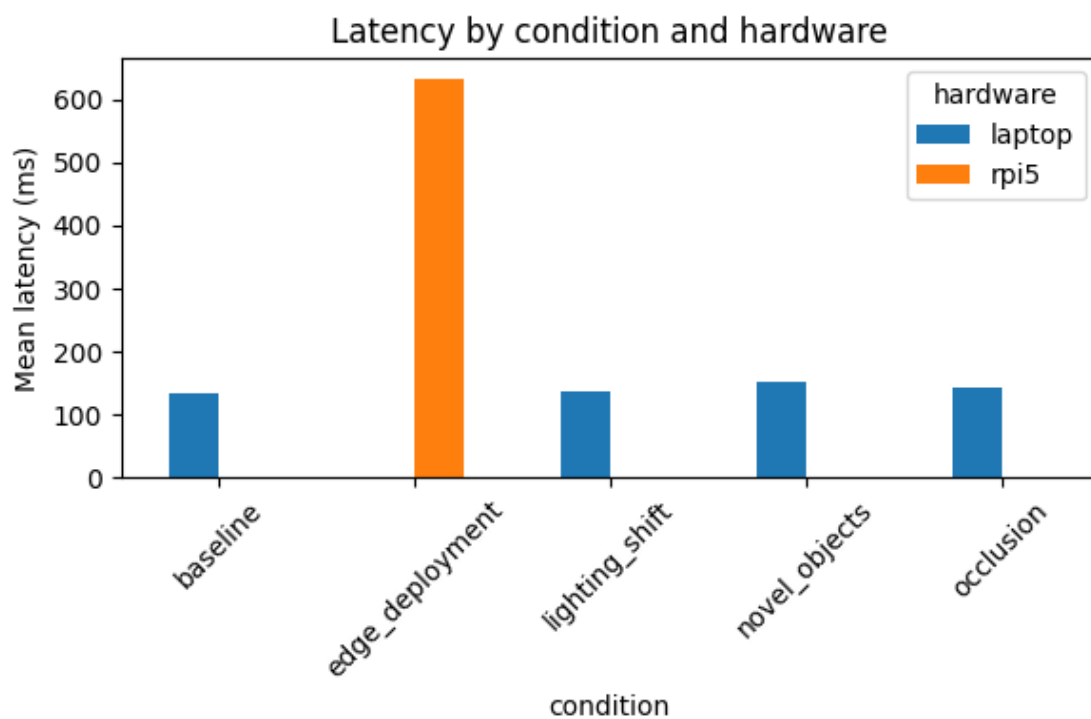
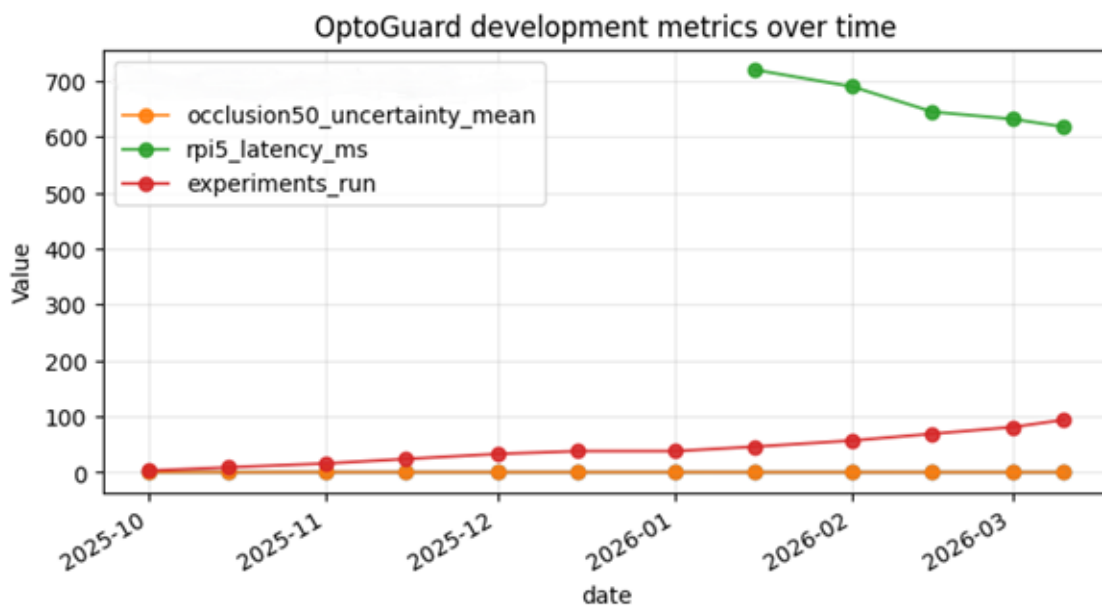


Figure 4. Development timeline (Oct 2025–Mar 2026)



8. References:

S. No.	Reference	Used for
1	Kumari, M., Vaddi, R., & Srijith, P. K., <i>Monte Carlo DropBlock for Modelling Uncertainty in Object Detection</i> , Pattern Recognition, 2024	Shows another method for estimating uncertainty in object detection (DropBlock). Mentioned as related work.
2	Kumaravelu, V., Gupta, S., & Srijith, P. K., <i>EvoCL: Continual Learning over Evolving Domains</i> , WACV 2025	Explains how models can adapt when environments change over time. Mentioned as future extension idea.
3	Gal, Y., & Ghahramani, Z., <i>Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning</i> , ICML 2016	Main paper explaining MC Dropout and why multiple dropout passes can estimate uncertainty.
4	Kendall, A., & Gal, Y., <i>What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?</i> , NeurIPS 2017	Explains types of uncertainty in vision models (epistemic vs aleatoric). Used to explain what your σ represents.
5	Tiago, M. et al., <i>Stochastic-YOLO: Efficient Probabilistic Object Detection under Dataset Shifts</i> , arXiv 2020	Example of applying uncertainty estimation to YOLO detectors. Used as related work comparison.